Cofinanziato dall'Unione europea

REGIONE PIEMONTE

DOCENTE	Wolfgang Cecchin
UNITA' FORMATIVA	Linux Server
ARGOMENTO	Firewall: iptables, ufw

Intervento realizzato da ITS TECNOLOGIE DELL'INFORMAZIONE E DELLA COMUNICAZIONE

Firewall in Linux

iptables e ufw

Cos'è un firewall

Un **firewall** è un sistema che controlla il traffico di rete in entrata e in uscita sulla base di un insieme di regole. Su un server Linux, il firewall è fondamentale per proteggere i servizi esposti: si decide esplicitamente cosa è permesso, e tutto il resto viene bloccato.

Il kernel Linux gestisce il filtraggio dei pacchetti attraverso un sottosistema chiamato `Netfilter`. Gli strumenti `iptables` e `ufw` sono semplicemente interfacce per scrivere regole che Netfilter applica.

> **Nota:** *Il successore moderno di iptables è nftables, diventato predefinito a partire da Debian 10. Su sistemi recenti, il comando iptables è in realtà un wrapper che traduce i comandi in regole nftables sottostanti. Per i server applicativi standard, iptables e ufw restano comunque lo strumento pratico di riferimento.*

Parte 1 — iptables

Come funziona iptables

`iptables` organizza le regole in **tabelle** e **catene**.

La tabella più usata è `filter`, che contiene tre catene predefinite:

- `INPUT` — traffico diretto al server stesso
- `OUTPUT` — traffico generato dal server
- `FORWARD` — traffico che transita attraverso il server (usato nei router)

Ogni pacchetto viene confrontato con le regole della catena, dall'alto verso il basso. Alla prima regola che corrisponde viene applicata l'**azione** (chiamata *target)*:

- `ACCEPT` — il pacchetto viene accettato
- `DROP` — il pacchetto viene scartato silenziosamente
- `REJECT` — il pacchetto viene rifiutato con un messaggio di errore

Se nessuna regola corrisponde, si applica la **policy di default** della catena.

Installazione

Su Debian è disponibile nei repository ufficiali:

```
sudo apt install iptables
```

Visualizzare le regole correnti

```
sudo iptables -L -v -n
```

- `-L` — elenca le regole
- `-v` — output verboso (mostra contatori, interfacce)
- `-n` — mostra IP e porte in formato numerico (non fa lookup DNS)

La sintassi di base

```
sudo iptables -A CATENA -p protocollo --dport porta -j TARGET
```

- `-A` — appende la regola in fondo alla catena (*append*)
- `-I` — inserisce la regola in cima (o in una posizione specifica)
- `-p` — protocollo (`tcp`, `udp`, `icmp`)
- `--dport` — porta di destinazione
- `-j` — azione da eseguire (*jump*)

Esempi pratici

Accettare le connessioni SSH (porta 22)

```
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
```

Accettare il traffico HTTP e HTTPS

```
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT
```

Accettare traffico sulla loopback

Indispensabile — senza questa regola molti servizi locali smettono di funzionare:

```
sudo iptables -A INPUT -i lo -j ACCEPT
```

Accettare connessioni già stabilite

Fondamentale: senza questa regola il server non riceve le risposte alle proprie richieste (aggiornamenti apt, query DNS, ecc.):

```
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
```

Bloccare un indirizzo IP specifico

```
sudo iptables -A INPUT -s 192.168.1.50 -j DROP
```

Permettere o bloccare il ping in entrata

```
# Permettere
sudo iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT

# Bloccare
sudo iptables -A INPUT -p icmp --icmp-type echo-request -j DROP
```

Se la policy di default è già DROP, la regola di blocco è ridondante — il ping verrebbe ignorato comunque. Diventa necessaria solo se la policy è ACCEPT e si vuole bloccare esplicitamente il ping lasciando aperto il resto.

Impostare la policy di default a DROP

Supponiamo di avere un server nginx con le regole precedenti già configurate. Si imposta la policy di default per bloccare tutto il resto:

```
sudo iptables -P INPUT DROP
```

Da questo momento il server accetta solo SSH, HTTP, HTTPS e le connessioni già stabilite. Tutto il resto viene silenziosamente ignorato.

Eliminare regole

Per eliminare una regola, si usa `-D` con la stessa sintassi usata per aggiungerla:

```
sudo iptables -D INPUT -p tcp --dport 80 -j ACCEPT
```

Oppure si elencano le regole con i numeri di riga e si elimina per numero:

```
sudo iptables -L INPUT --line-numbers
sudo iptables -D INPUT 3
```

Svuotare tutte le regole (flush)

Durante le prove in laboratorio, per ripartire da zero:

```
sudo iptables -F
sudo iptables -P INPUT ACCEPT
sudo iptables -P OUTPUT ACCEPT
sudo iptables -P FORWARD ACCEPT
```

Configurazione tramite file di regole

Invece di digitare le regole una per una, è possibile scriverle in un file di testo e caricarle tutte in una volta. Questo è il formato usato anche da `iptables-persistent` internamente.

Un esempio di file di configurazione tipico per un server:

```
*filter
:INPUT DROP
:FORWARD DROP
:OUTPUT ACCEPT

-A INPUT -i lo -s 127.0.0.1 -d 127.0.0.1 -j ACCEPT
-A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
-A INPUT -p icmp --icmp-type echo-request -j ACCEPT

COMMIT
```

Vediamo cosa fa ogni parte:

Le policy di default

```
:INPUT DROP
:FORWARD DROP
:OUTPUT ACCEPT
```

Imposta il comportamento per i pacchetti che non corrispondono a nessuna regola. INPUT e FORWARD vengono scartati (approccio *deny by default* — il più sicuro), mentre il traffico in uscita è libero.

Le regole

```
-A INPUT -i lo -s 127.0.0.1 -d 127.0.0.1 -j ACCEPT
```

Permette il traffico sulla loopback. Necessario per i servizi locali che comunicano tra loro, ad esempio PHP che si connette a MySQL su `127.0.0.1`.

```
-A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
```

Permette le risposte a connessioni già avviate dal server. Senza questa regola, il server non riceverebbe nessuna risposta alle proprie richieste (aggiornamenti apt, query DNS, ecc.).

```
-A INPUT -p icmp --icmp-type echo-request -j ACCEPT
```

Permette il ping in entrata. Utile per verificare che il server sia raggiungibile dalla rete.

Caricare il file

Il file viene salvato in `/etc/iptables.rules` e caricato con:

```
sudo iptables-restore < /etc/iptables.rules
```

È equivalente a digitare ogni regola a mano, ma più rapido e riproducibile. Su questo formato si basa anche la persistenza automatica descritta nella sezione seguente.

Persistenza delle regole

Le regole `iptables` **non sopravvivono al riavvio**. Vengono caricate in memoria e al riavvio si azzerano. Per renderle persistenti su Debian si installa:

```
sudo apt install iptables-persistent
```

Le regole vengono salvate in `/etc/iptables/rules.v4` (IPv4) e `/etc/iptables/rules.v6` (IPv6). Per salvare lo stato corrente:

```
sudo netfilter-persistent save
```

Parte 2 — ufw

Cos'è ufw

`ufw` (*Uncomplicated Firewall*) è un frontend per `iptables` progettato per semplificare la gestione del firewall. Invece di ragionare in termini di catene e target, si lavora con concetti più diretti: *abilita questa porta*, *blocca questo IP*. Dietro le quinte, ufw scrive comunque regole iptables, ma non è necessario gestirle a mano.

Installazione e stato

```
sudo apt install ufw
```

Per vedere lo stato attuale:

```
sudo ufw status
```

Se ufw è disabilitato, l'output sarà semplicemente `Status: inactive`.

Per uno stato più dettagliato con le regole numerate:

```
sudo ufw status numbered
sudo ufw status verbose
```

Abilitare e disabilitare

```
sudo ufw enable
sudo ufw disable
```

> **Attenzione:** *Prima di abilitare ufw su un server remoto, assicurarsi di aver aggiunto la regola per SSH. Altrimenti ci si taglia fuori dalla macchina.*

Aggiungere regole

Permettere una porta

```
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
```

ufw conosce i servizi più comuni per nome:

```
sudo ufw allow ssh
sudo ufw allow http
sudo ufw allow https
```

Permettere un profilo applicazione

Alcuni pacchetti (nginx, OpenSSH, ecc.) installano profili ufw predefiniti in `/etc/ufw/applications.d/`. Si può usarli direttamente:

```
sudo ufw allow 'Nginx Full'    # permette sia 80 che 443
sudo ufw allow 'OpenSSH'
```

Per vedere i profili disponibili:

```
sudo ufw app list
```

Bloccare un indirizzo IP

```
sudo ufw deny from 192.168.1.50
```

Permettere l'accesso solo da una subnet specifica

Scenario pratico: un database MySQL che deve essere raggiungibile solo dai server nella rete interna `192.168.10.0/24`, non dall'esterno.

```
sudo ufw allow from 192.168.10.0/24 to any port 3306
```

Permettere una connessione su una specifica interfaccia di rete

```
sudo ufw allow in on eth0 to any port 80
```

Eliminare regole

Con il numero di riga (da `ufw status numbered`):

```
sudo ufw delete 3
```

Oppure con la stessa sintassi usata per creare la regola:

```
sudo ufw delete allow 80/tcp
```

Policy di default

La configurazione tipica per un server:

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
```

Si nega tutto il traffico in entrata e si aprono solo le porte necessarie, mentre si lascia libero il traffico in uscita.

Logging

ufw può registrare nei log di sistema le connessioni bloccate o accettate:

```
sudo ufw logging on
```

I livelli disponibili sono `low`, `medium`, `high`, `full`. Il default è `low`.

I log vengono scritti in `/var/log/ufw.log` e sono leggibili anche con:

```
sudo journalctl | grep UFW
```

Reset

Per riportare ufw allo stato iniziale, eliminando tutte le regole:

```
sudo ufw reset
```

---

iptables vs ufw — quando usare quale

	iptables	ufw
Controllo	Granulare, completo	Semplificato
Curva di apprendimento	Ripida	Bassa
Uso tipico	Server avanzati, script, router	Server applicativi standard
Persistenza	Manuale (iptables-persistent)	Automatica
Leggibilità regole	Bassa	Alta

In un contesto professionale, `iptables` viene usato quando si ha bisogno di controllo fine — ad esempio per NAT, port forwarding, o regole complesse su firewall di rete. `ufw` è la scelta naturale per un server applicativo standard dove si vogliono semplicemente aprire le porte necessarie e bloccare tutto il resto.